

✓ Creating Numbers/images with AI: A Hands-on Diffusion Model Exercise

Introduction

In this assignment, you'll learn how to create an AI model that can generate realistic images from scratch using a powerful technique called 'diffusion'. Think of it like teaching AI to draw by first learning how images get blurry and then learning to make them clear again.

What We'll Build

- A diffusion model capable of generating realistic images
- For most students: An AI that generates handwritten digits (0-9) using the MNIST dataset
- For students with more computational resources: Options to work with more complex datasets
- Visual demonstrations of how random noise gradually transforms into clear, recognizable images
- By the end, your AI should create images realistic enough for another AI to recognize them

Dataset Options

This lab offers flexibility based on your available computational resources:

- Standard Option (Free Colab): We'll primarily use the MNIST handwritten digit dataset, which works well with limited GPU memory and completes training in a reasonable time frame. Most examples and code in this notebook are optimized for MNIST.
- Advanced Option: If you have access to more powerful GPUs (either through Colab Pro/Pro+ or your own hardware), you can experiment with more complex datasets like Fashion-MNIST, CIFAR-10, or even face generation. You'll need to adapt the model architecture, hyperparameters, and evaluation metrics accordingly.

Resource Requirements

- Basic MNIST: Works with free Colab GPUs (2-4GB VRAM), ~30 minutes training
- Fashion-MNIST: Similar requirements to MNIST CIFAR-10: Requires more memory (8-12GB VRAM) and longer training (~2 hours)
- Higher resolution images: Requires substantial GPU resources and several hours of training

Before You Start

1. Make sure you're running this in Google Colab or another environment with GPU access
2. Go to 'Runtime' → 'Change runtime type' and select 'GPU' as your hardware accelerator
3. Each code cell has comments explaining what it does
4. Don't worry if you don't understand every detail - focus on the big picture!
5. If working with larger datasets, monitor your GPU memory usage carefully

The concepts you learn with MNIST will scale to more complex datasets, so even if you're using the basic option, you'll gain valuable knowledge about generative AI that applies to more advanced applications.

✓ Step 1: Setting Up Our Tools

Assigned to: Jonah Joseph

First, let's install and import all the tools we need. Run this cell and wait for it to complete.

```
# Step 1: Install required packages
%pip install einops
print("Package installation complete.")

# Step 2: Import libraries
# --- Core PyTorch libraries ---
import torch # Main deep learning framework
import torch.nn.functional as F # Neural network functions like activation functions
import torch.nn as nn # Neural network building blocks (layers)
from torch.optim import Adam # Optimization algorithm for training

# --- Data handling ---
from torch.utils.data import Dataset, DataLoader # For organizing and loading our data
import torchvision # Library for computer vision datasets and models
import torchvision.transforms as transforms # For preprocessing images
```

```
# --- Tensor manipulation ---
import random # For random operations
from einops.layers.torch import Rearrange # For reshaping tensors in neural networks
from einops import rearrange # For elegant tensor reshaping operations
import numpy as np # For numerical operations on arrays

# --- System utilities ---
import os # For operating system interactions (used for CPU count)

# --- Visualization tools ---
import matplotlib.pyplot as plt # For plotting images and graphs
from PIL import Image # For image processing
from torchvision.utils import save_image, make_grid # For saving and displaying image grids

# Step 3: Set up device (GPU or CPU)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"We'll be using: {device}")

# Check if we're actually using GPU (for students to verify)
if device.type == "cuda":
    print(f"GPU name: {torch.cuda.get_device_name(0)}")
    print(f"GPU memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
else:
    print("Note: Training will be much slower on CPU. Consider using Google Colab with GPU enabled.")
```

```
Requirement already satisfied: einops in /usr/local/lib/python3.13/dist-packages (0.8.2)
Package installation complete.
We'll be using: cuda
GPU name: Tesla T4
GPU memory: 15.64 GB
```

✓ REPRODUCIBILITY AND DEVICE SETUP

```
# Step 4: Set random seeds for reproducibility
# Diffusion models are sensitive to initialization, so reproducible results help with debugging
SEED = 42 # Universal seed value for reproducibility
torch.manual_seed(SEED) # PyTorch random number generator
np.random.seed(SEED) # NumPy random number generator
random.seed(SEED) # Python's built-in random number generator

print(f"Random seeds set to {SEED} for reproducible results")

# Configure CUDA for GPU operations if available
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED) # GPU random number generator
    torch.cuda.manual_seed_all(SEED) # All GPUs random number generator

# Ensure deterministic GPU operations
# Note: This slightly reduces performance but ensures results are reproducible
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

try:
    # Check available GPU memory
    gpu_memory = torch.cuda.get_device_properties(0).total_memory / 1e9 # Convert to GB
    print(f"Available GPU Memory: {gpu_memory:.1f} GB")

    # Add recommendation based on memory
    if gpu_memory < 4:
        print("Warning: Low GPU memory. Consider reducing batch size if you encounter OOM errors.")
    except Exception as e:
        print(f"Could not check GPU memory: {e}")
else:
    print("No GPU detected. Training will be much slower on CPU.")
    print("If you're using Colab, go to Runtime > Change runtime type and select GPU.")
```

```
Random seeds set to 42 for reproducible results
Available GPU Memory: 15.6 GB
```

✓ Step 2: Choosing Your Dataset

Elijah Raines

You have several options for this exercise, depending on your computer's capabilities:

Option 1: MNIST (Basic - Works on Free Colab)

- Content: Handwritten digits (0-9)
- Image size: 28x28 pixels, Grayscale
- Training samples: 60,000
- Memory needed: ~2GB GPU
- Training time: ~15-30 minutes on Colab
- **Choose this if:** You're using free Colab or have a basic GPU

Option 2: Fashion-MNIST (Intermediate)

- Content: Clothing items (shirts, shoes, etc.)
- Image size: 28x28 pixels, Grayscale
- Training samples: 60,000
- Memory needed: ~2GB GPU
- Training time: ~15-30 minutes on Colab
- **Choose this if:** You want more interesting images but have limited GPU

Option 3: CIFAR-10 (Advanced)

- Content: Real-world objects (cars, animals, etc.)
- Image size: 32x32 pixels, Color (RGB)
- Training samples: 50,000
- Memory needed: ~4GB GPU
- Training time: ~1-2 hours on Colab
- **Choose this if:** You have Colab Pro or a good local GPU (8GB+ memory)

Option 4: CelebA (Expert)

- Content: Celebrity face images
- Image size: 64x64 pixels, Color (RGB)
- Training samples: 200,000
- Memory needed: ~8GB GPU
- Training time: ~3-4 hours on Colab
- **Choose this if:** You have excellent GPU (12GB+ memory)

To use your chosen dataset, uncomment its section in the code below and make sure all others are commented out.

```
#####
# SECTION 2: DATASET SELECTION AND CONFIGURATION
#####
# STUDENT INSTRUCTIONS:
# 1. Choose ONE dataset option based on your available GPU memory
# 2. Uncomment ONLY ONE dataset section below
# 3. Make sure all other dataset sections remain commented out

#-----
# OPTION 1: MNIST (Basic - 2GB GPU)
#-----
# Recommended for: Free Colab or basic GPU
# Memory needed: ~2GB GPU
# Training time: ~15-30 minutes

# IMG_SIZE = 28
# IMG_CH = 1
# N_CLASSES = 10
# BATCH_SIZE = 64
# EPOCHS = 30

# transform = transforms.Compose([
#     transforms.ToTensor(),
#     transforms.Normalize((0.5,), (0.5,))
# ])

# Your code to load the MNIST dataset
# Hint: Use torchvision.datasets.MNIST with root='./data', train=True,
```

```

#         transform=transform, and download=True
# Then print a success message

# Enter your code here:

#-----
# OPTION 2: Fashion-MNIST (Intermediate - 2GB GPU)
#-----
# Uncomment this section to use Fashion-MNIST instead
IMG_SIZE = 28
IMG_CH = 1
N_CLASSES = 10
BATCH_SIZE = 64
EPOCHS = 30

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Your code to load the Fashion-MNIST dataset
# Hint: Very similar to MNIST but use torchvision.datasets.FashionMNIST

# Enter your code here:
dataset = torchvision.datasets.FashionMNIST(
    root='./data', train=True, transform=transform, download=True
)
print(f"Fashion-MNIST loaded successfully: {len(dataset):,} training samples, {N_CLASSES} classes.")

#-----
# OPTION 3: CIFAR-10 (Advanced - 4GB+ GPU)
#-----
# Uncomment this section to use CIFAR-10 instead
# IMG_SIZE = 32
# IMG_CH = 3
# N_CLASSES = 10
# BATCH_SIZE = 32 # Reduced batch size for memory
# EPOCHS = 50      # More epochs for complex data

# Your code to create the transform and load CIFAR-10
# Hint: Use transforms.Normalize with RGB means and stds ((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
# Then load torchvision.datasets.CIFAR10

# Enter your code here:

```

Fashion-MNIST loaded successfully: 60,000 training samples, 10 classes.

```

#Validating Dataset Selection
#Let's add code to validate that a dataset was selected
# and check if your GPU has enough memory:

# Validate dataset selection
if 'dataset' not in locals():
    raise ValueError("""
    ✖ ERROR: No dataset selected! Please uncomment exactly one dataset option.
    Available options:
    1. MNIST (Basic) - 2GB GPU
    2. Fashion-MNIST (Intermediate) - 2GB GPU
    3. CIFAR-10 (Advanced) - 4GB+ GPU
    4. CelebA (Expert) - 8GB+ GPU
    """)

# Your code to validate GPU memory requirements
# Hint: Check torch.cuda.is_available() and use torch.cuda.get_device_properties(0).total_memory
# to get available GPU memory, then compare with dataset requirements

# Enter your code here:
if not isinstance(dataset, torchvision.datasets.FashionMNIST):
    raise ValueError("Please select Option 2: Fashion-MNIST and rerun the dataset cell.")
if len(dataset) != 60000 or len(dataset.classes) != N_CLASSES:
    raise ValueError("Unexpected Fashion-MNIST dataset size or class count.")
print("Dataset selection validated: Fashion-MNIST.")

required_memory = 2.0 # Fashion-MNIST needs approximately 2GB GPU memory

```

```

if torch.cuda.is_available():
    gpu_memory = torch.cuda.get_device_properties(0).total_memory / 1e9
    free_memory = torch.cuda.mem_get_info(0)[0] / 1e9
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"GPU memory: {gpu_memory:.2f} GB total, {free_memory:.2f} GB free")
    if min(gpu_memory, free_memory) < required_memory:
        print("Warning: Less than 2GB GPU memory is available. Reduce BATCH_SIZE if training runs out of memory.")
    else:
        print("GPU memory check passed: sufficient memory for Fashion-MNIST.")
else:
    print("No GPU detected. Dataset loading works on CPU, but training will be much slower.")
    print("For GPU training, go to Runtime > Change runtime type and select GPU.")

```

Dataset selection validated: Fashion-MNIST.
GPU: Tesla T4
GPU memory: 15.64 GB total, 15.04 GB free
GPU memory check passed: sufficient memory for Fashion-MNIST.

```

#Dataset Properties and Data Loaders
#Now let's examine our dataset
#and set up the data loaders:

# Your code to check sample batch properties
# Hint: Get a sample batch using next(iter(DataLoader(dataset, batch_size=1)))
# Then print information about the dataset shape, type, and value ranges

# Enter your code here:
sample_batch = next(iter(DataLoader(dataset, batch_size=1, shuffle=False)))
sample_images, sample_labels = sample_batch
print(f"Dataset: {type(dataset).__name__}, {len(dataset):,} samples")
print(f"Classes: {dataset.classes}")
print(f"Image batch shape: {sample_images.shape}")
print(f"Image type: {type(sample_images).__name__}, dtype: {sample_images.dtype}")
print(f"Image value range: [{sample_images.min().item():.2f}, {sample_images.max().item():.2f}]")
print(f"Label shape: {sample_labels.shape}, dtype: {sample_labels.dtype}")
print(f"Sample label: {sample_labels.item()} ({dataset.classes[sample_labels.item()]})")
print(f"Dataset label range: [{dataset.targets.min().item()}, {dataset.targets.max().item()}]")
assert tuple(sample_images.shape[1:]) == (IMG_CH, IMG_SIZE, IMG_SIZE)
assert torch.isfinite(sample_images).all()
assert sample_images.min() >= -1 and sample_images.max() <= 1

#=====
# SECTION 3: DATASET SPLITTING AND DATALOADER CONFIGURATION
#=====
# Create train-validation split

# Your code to create a train-validation split (80% train, 20% validation)
# Hint: Use random_split() with appropriate train_size and val_size
# Be sure to use a fixed generator for reproducibility

# Enter your code here:
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(
    dataset, [train_size, val_size], generator=torch.Generator().manual_seed(SEED)
)
print(f"Train-validation split: {len(train_dataset):,} training, {len(val_dataset):,} validation samples (80/20).")

# Your code to create dataloaders for training and validation
# Hint: Use DataLoader with batch_size=BATCH_SIZE, appropriate shuffle settings,
# and num_workers based on available CPU cores

# Enter your code here:
num_workers = min(2, os.cpu_count() or 1)
train_dataloader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE, shuffle=True,
    num_workers=num_workers, pin_memory=torch.cuda.is_available(),
    generator=torch.Generator().manual_seed(SEED)
)
val_dataloader = DataLoader(
    val_dataset, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=num_workers, pin_memory=torch.cuda.is_available()
)

```

```

)
print(f"DataLoaders ready: {len(train_dataloader)} training batches, {len(val_dataloader)} validation batches.")
print(f"Batch size: {BATCH_SIZE}, workers: {num_workers}")
train_images, train_labels = next(iter(train_dataloader))
val_images, val_labels = next(iter(val_dataloader))
print(f"Training batch: images {train_images.shape}, labels {train_labels.shape}")
print(f"Validation batch: images {val_images.shape}, labels {val_labels.shape}")

```

```

Dataset: FashionMNIST, 60,000 samples
Classes: ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
Image batch shape: torch.Size([1, 1, 28, 28])
Image type: Tensor, dtype: torch.float32
Image value range: [-1.00, 1.00]
Label shape: torch.Size([1]), dtype: torch.int64
Sample label: 9 (Ankle boot)
Dataset label range: [0, 9]
Train-validation split: 48,000 training, 12,000 validation samples (80/20).
DataLoaders ready: 750 training batches, 188 validation batches.
Batch size: 64, workers: 2
Training batch: images torch.Size([64, 1, 28, 28]), labels torch.Size([64])
Validation batch: images torch.Size([64, 1, 28, 28]), labels torch.Size([64])

```

▼ Step 3: Building Our Model Components

Assigned to: Samantha Mireles

Now we'll create the building blocks of our AI model. Think of these like LEGO pieces that we'll put together to make our number generator:

- GELUConvBlock: The basic building block that processes images
- DownBlock: Makes images smaller while finding important features
- UpBlock: Makes images bigger again while keeping the important features
- Other blocks: Help the model understand time and what number to generate

```

import torch.nn as nn

# Basic building block that processes images
class GELUConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, group_size):
        """
        Creates a block with convolution, normalization, and activation

        Args:
            in_ch (int): Number of input channels
            out_ch (int): Number of output channels
            group_size (int): Number of groups for GroupNorm
        """
        super().__init__()

        # Check that group_size is compatible with out_ch
        if out_ch % group_size != 0:
            print(f"Warning: out_ch ({out_ch}) is not divisible by group_size ({group_size})")
            # Adjust group_size to be compatible
            group_size = min(group_size, out_ch)
            while out_ch % group_size != 0:
                group_size -= 1
            print(f"Adjusted group_size to {group_size}")

        # Your code to create layers for the block
        # Hint: Use nn.Conv2d, nn.GroupNorm, and nn.GELU activation
        # Then combine them using nn.Sequential

        # Enter your code here:
        self.model = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3, stride=1, padding=1),
            nn.GroupNorm(group_size, out_ch),
            nn.GELU()
        )

    def forward(self, x):
        # Your code for the forward pass
        # Hint: Simply pass the input through the model

```

```
# Enter your code here:
return self.model(x)
```

```
# Rearranges pixels to downsample the image (2x reduction in spatial dimensions)
# Rearranges pixels to downsample the image (2x reduction in spatial dimensions)
class RearrangePoolBlock(nn.Module):
    def __init__(self, in_chs, group_size):
        """
        Downsamples the spatial dimensions by 2x while preserving information
        """
        super().__init__()

        self.rearrange = Rearrange(
            "b c (h p1) (w p2) -> b (c p1 p2) h w",
            p1=2,
            p2=2
        )

        self.conv = GELUConvBlock(
            4 * in_chs,
            in_chs,
            group_size
        )

    def forward(self, x):
        x = self.rearrange(x)
        return self.conv(x)
```

```
#Let's implement the upsampling block for our U-Net architecture:
class DownBlock(nn.Module):
    """
    Downsampling block for encoding path in U-Net architecture.

    This block:
    1. Processes input features with two convolutional blocks
    2. Downsamples spatial dimensions by 2x using pixel rearrangement

    Args:
        in_chs (int): Number of input channels
        out_chs (int): Number of output channels
        group_size (int): Number of groups for GroupNorm
    """
    def __init__(self, in_chs, out_chs, group_size):
        super().__init__() # Simplified super() call, equivalent to original

        # Sequential processing of features
        layers = [
            GELUConvBlock(in_chs, out_chs, group_size), # First conv block changes channel dimensions
            GELUConvBlock(out_chs, out_chs, group_size), # Second conv block processes features
            RearrangePoolBlock(out_chs, group_size)      # Downsampling (spatial dims: H,W → H/2,W/2)
        ]
        self.model = nn.Sequential(*layers)

        # Log the configuration for debugging
        print(f"Created DownBlock: in_chs={in_chs}, out_chs={out_chs}, spatial_reduction=2x")

    def forward(self, x):
        """
        Forward pass through the DownBlock.

        Args:
            x (torch.Tensor): Input tensor of shape [B, in_chs, H, W]

        Returns:
            torch.Tensor: Output tensor of shape [B, out_chs, H/2, W/2]
        """
        return self.model(x)
```

```
#Now let's implement the upsampling block for our U-Net architecture:
#Now let's implement the upsampling block for our U-Net architecture:
class UpBlock(nn.Module):
    """
    Upsampling block for decoding path in U-Net architecture.
```

This block:

1. Takes features from the decoding path and corresponding skip connection
2. Concatenates them along the channel dimension
3. Upsamples spatial dimensions by 2x using transposed convolution
4. Processes features through multiple convolutional blocks

```
def __init__(self, in_chs, out_chs, group_size):
    super().__init__()

    self.up = nn.ConvTranspose2d(
        2 * in_chs,
        out_chs,
        kernel_size=2,
        stride=2
    )

    self.conv = nn.Sequential(
        GELUConvBlock(out_chs, out_chs, group_size),
        GELUConvBlock(out_chs, out_chs, group_size),
        GELUConvBlock(out_chs, out_chs, group_size),
        GELUConvBlock(out_chs, out_chs, group_size)
    )

    print(
        f"Created UpBlock: in_chs={in_chs}, "
        f"out_chs={out_chs}, spatial_increas=2x"
    )

def forward(self, x, skip):
    x = torch.cat((x, skip), dim=1)
    x = self.up(x)
    x = self.conv(x)
    return x
```

Here we implement the time embedding block for our U-Net architecture:

Helps the model understand time steps in diffusion process

```
class SinusoidalPositionEmbedBlock(nn.Module):
```

```
    """
```

Creates sinusoidal embeddings for time steps in diffusion process.

This embedding scheme is adapted from the Transformer architecture and provides a unique representation for each time step that preserves relative distance information.

Args:

dim (int): Embedding dimension

```
    """
```

```
def __init__(self, dim):
```

```
    super().__init__()
```

```
    self.dim = dim
```

```
def forward(self, time):
```

```
    """
```

Computes sinusoidal embeddings for given time steps.

Args:

time (torch.Tensor): Time steps tensor of shape [batch_size]

Returns:

torch.Tensor: Time embeddings of shape [batch_size, dim]

```
    """
```

```
    device = time.device
```

```
    half_dim = self.dim // 2
```

```
    embeddings = torch.log(torch.tensor(10000.0, device=device)) / (half_dim - 1)
```

```
    embeddings = torch.exp(torch.arange(half_dim, device=device) * -embeddings)
```

```
    embeddings = time[:, None] * embeddings[None, :]
```

```
    embeddings = torch.cat((embeddings.sin(), embeddings.cos()), dim=-1)
```

```
    return embeddings
```

Helps the model understand which number/image to draw (class conditioning)

```
class EmbedBlock(nn.Module):
```

```

"""
Creates embeddings for class conditioning in diffusion models.

This module transforms a one-hot or index representation of a class
into a rich embedding that can be added to feature maps.

Args:
    input_dim (int): Input dimension (typically number of classes)
    emb_dim (int): Output embedding dimension
"""
def __init__(self, input_dim, emb_dim):
    super(EmbedBlock, self).__init__()
    self.input_dim = input_dim

    # Your code to create the embedding layers
    # Hint: Use nn.Linear layers with a GELU activation, followed by
    # nn.Unflatten to reshape for broadcasting with feature maps

    # Enter your code here:
    self.model = nn.Sequential(
nn.Linear(input_dim, emb_dim),
nn.GELU(),
nn.Linear(emb_dim, emb_dim),
nn.Unflatten(1, (emb_dim, 1, 1))
)

def forward(self, x):
    """
    Computes class embeddings for the given class indices.

    Args:
        x (torch.Tensor): Class indices or one-hot encodings [batch_size, input_dim]

    Returns:
        torch.Tensor: Class embeddings of shape [batch_size, emb_dim, 1, 1]
                        (ready to be added to feature maps)
    """
    x = x.view(-1, self.input_dim)
    return self.model(x)

```

```

# Main U-Net model that puts everything together
# Main U-Net model that puts everything together
class UNet(nn.Module):

    def __init__(self, T, img_ch, img_size, down_chs, t_embed_dim, c_embed_dim):
        super().__init__()

        self.T = T
        group_size = 8

        # Time embedding
        self.time_embed = nn.Sequential(
            SinusoidalPositionEmbedBlock(t_embed_dim),
            nn.Linear(t_embed_dim, down_chs[-1]),
            nn.GELU()
        )

        # Class embedding
        self.class_embed = EmbedBlock(
            c_embed_dim,
            down_chs[-1]
        )

        # Initial convolution
        self.init_conv = GELUConvBlock(
            img_ch,
            down_chs[0],
            group_size
        )

        # Downsampling path
        self.down_blocks = nn.ModuleList([

```

```

        DownBlock(
            down_chs[i],
            down_chs[i + 1],
            group_size
        )
    for i in range(len(down_chs) - 1)
])

# Middle blocks
self.mid_blocks = nn.Sequential(
    GELUConvBlock(
        down_chs[-1],
        down_chs[-1],
        group_size
    ),
    GELUConvBlock(
        down_chs[-1],
        down_chs[-1],
        group_size
    )
)

# Upsampling path
up_chs = list(reversed(down_chs))

self.up_blocks = nn.ModuleList([
    UpBlock(
        up_chs[i],
        up_chs[i + 1],
        group_size
    )
    for i in range(len(up_chs) - 1)
])

# Final convolution
self.final_conv = nn.Conv2d(
    down_chs[0],
    img_ch,
    kernel_size=1
)

print(f"Created UNet with {len(down_chs)} scale levels")
print(f"Channel dimensions: {down_chs}")

def forward(self, x, t, c, c_mask):

    # Time embedding
    t_emb = self.time_embed(t.float() / self.T)
    t_emb = t_emb.unsqueeze(-1).unsqueeze(-1)

    # Class embedding
    c_emb = self.class_embed(c)
    c_mask = c_mask.view(-1, 1, 1, 1).float()
    c_emb = c_emb * c_mask

    # Initial feature extraction
    x = self.init_conv(x)

    # Downsampling path
    skips = []

    for down in self.down_blocks:
        x = down(x)
        skips.append(x)

    # Middle processing
    x = self.mid_blocks(x)
    x = x + t_emb + c_emb

    # Upsampling path
    for up, skip in zip(self.up_blocks, reversed(skips)):
        x = up(x, skip)

    # Final projection
    x = self.final_conv(x)

    return x

```

✓ Step 4: Setting Up The Diffusion Process

Assigned to: Brandon M

Now we'll create the process of adding and removing noise from images. Think of it like:

1. Adding fog: Slowly making the image more and more blurry until you can't see it
2. Removing fog: Teaching the AI to gradually make the image clearer
3. Controlling the process: Making sure we can generate specific numbers we want

```
# Set up the noise schedule
n_steps = 100 # How many steps to go from clear image to noise
beta_start = 0.0001 # Starting noise level (small)
beta_end = 0.02 # Ending noise level (larger)

# Create schedule of gradually increasing noise levels
beta = torch.linspace(beta_start, beta_end, n_steps).to(device)

# Calculate important values used in diffusion equations
alpha = 1 - beta # Portion of original image to keep at each step
alpha_bar = torch.cumprod(alpha, dim=0) # Cumulative product of alphas
sqrt_alpha_bar = torch.sqrt(alpha_bar) # For scaling the original image
sqrt_one_minus_alpha_bar = torch.sqrt(1 - alpha_bar) # For scaling the noise
```

```
# Function to add noise to images (forward diffusion process)
def add_noise(x_0, t):
    """
    Add noise to images according to the forward diffusion process.

    The formula is:  $x_t = \sqrt{\alpha_t} * x_0 + \sqrt{1-\alpha_t} * \epsilon$ 
    where  $\epsilon$  is random noise and  $\alpha_t$  is the cumulative product of  $(1-\beta)$ .
    """
    # Create random Gaussian noise with same shape as image
    noise = torch.randn_like(x_0)

    # Get noise schedule values for the specified timesteps
    # Reshape to allow broadcasting with image dimensions
    sqrt_alpha_bar_t = sqrt_alpha_bar[t].reshape(-1, 1, 1, 1)
    sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alpha_bar[t].reshape(-1, 1, 1, 1)

    # Apply the forward diffusion equation:
    # Mixture of original image (scaled down) and noise (scaled up)
    x_t = sqrt_alpha_bar_t * x_0 + sqrt_one_minus_alpha_bar_t * noise

    return x_t, noise
```

```
@torch.no_grad()
def remove_noise(x_t, t, model, c, c_mask):
    """
    Performs a single reverse diffusion step to remove noise from an image.
    """

    # 1. PREDICT NOISE
    # Pass the noisy image and class label through the U-Net

    pred_noise = model(x_t, t, c, c_mask)

    # 2. EXTRACT SCHEDULE PARAMETERS
    # Grab alpha, alpha_bar, and beta for current timestep 't'

    alpha_t = alpha[t].reshape(-1, 1, 1, 1)
    beta_t = beta[t].reshape(-1, 1, 1, 1)
    sqrt_1_minus_alpha_bar_t = sqrt_one_minus_alpha_bar[t].reshape(-1, 1, 1, 1)

    # 3. CALCULATE DENOISED MEAN
    # Estimate the mean of the image at timestep t-1

    mean = (1 / torch.sqrt(alpha_t)) * (
```

```

    x_t = (beta_t / sqrt(1 - alpha_bar_t)) * pred_noise
)

# 4. ADD NOISE OR RETURN FINAL IMAGE
# At t=0, the image is clean, so we return the mean directly

if t[0] == 0:
    return mean

# For t > 0, add random noise scaled by beta to maintain stochasticity

noise = torch.randn_like(x_t)
return mean + torch.sqrt(beta_t) * noise

# Function to visualize how noise progressively degrades an image

def show_noise_progression(image, num_steps=5):
    """
    Plots an original clean image alongside versions with increasing noise levels.
    """
    plt.figure(figsize=(15, 3))

    # -----

    # 1. DISPLAY THE ORIGINAL CLEAN IMAGE

    plt.subplot(1, num_steps, 1)

    if IMG_CH == 1:
        # Render single-channel grayscale image (e.g., MNIST / Fashion-MNIST)
        # Added interpolation='bicubic' to smooth pixelation from 28x28 display scaling
        plt.imshow(image[0].cpu(), cmap='gray', interpolation='bicubic')
    else:
        # Convert multi-channel RGB image format [C, H, W] -> [H, W, C]
        img = image.permute(1, 2, 0).cpu()
        if img.min() < 0:
            img = (img + 1) / 2 # Rescale values from [-1, 1] to [0, 1]
        plt.imshow(img, interpolation='bicubic')

    plt.title('Original')
    plt.axis('off')

    # -----

    # 2. GENERATE AND DISPLAY NOISY VERSIONS

    for i in range(1, num_steps):
        # Calculate target timestep based on percentage progress
        t_idx = int((i / num_steps) * n_steps)
        t = torch.tensor([t_idx]).to(device)

        # Apply forward diffusion to corrupt the clean image
        noisy_image, _ = add_noise(image.unsqueeze(0), t)

        # Plot noisy sample in its respective grid position
        plt.subplot(1, num_steps, i + 1)

        if IMG_CH == 1:
            plt.imshow(noisy_image[0][0].cpu(), cmap='gray', interpolation='bicubic')
        else:
            img = noisy_image[0].permute(1, 2, 0).cpu()
            if img.min() < 0:
                img = (img + 1) / 2
            plt.imshow(img, interpolation='bicubic')

        plt.title(f'{int((i / num_steps) * 100)}% Noise')
        plt.axis('off')

    plt.show()

    # -----

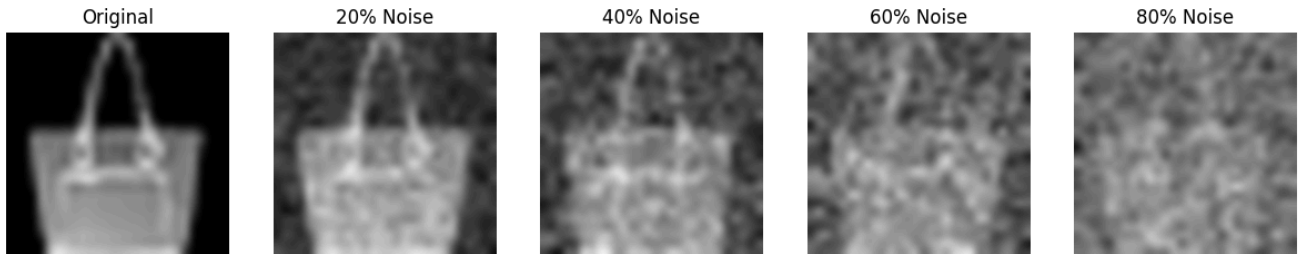
    # 3. RUN VISUALIZATION ON A REAL SAMPLE
    # Fetch the first sample image from the training dataloader

    sample_batch = next(iter(train_dataloader))

```

```
sample_image = sample_batch[0][0].to(device)

# Execute progression display
show_noise_progression(sample_image)
```



Step 4 Reflection - Brandon M

Setting Up the Diffusion Process

In Step 4, I set up the core math behind the forward and reverse diffusion processes. My main job was filling in the `add_noise` function so it could take a clean image and blend it with Gaussian noise based on our timestep schedule. I also cleaned up the reverse diffusion `remove_noise` function, which uses our U-Net model to predict that noise and step-by-step strip it away to rebuild the image. Finally, I worked on the `show_noise_progression` visualization cell so we could clearly watch a clean image gradually get covered in noise from 0% all the way up to 80%.

The main issue I hit early on was a `NameError: name 'x_t' is not defined` when running the visualization cell. That happened because `x_t` wasn't being defined inside `add_noise` before the return statement. I fixed it by filling in the forward diffusion formula (`x_t = sqrt_alpha_bar_t * x_0 + sqrt_one_minus_alpha_bar_t * noise`) and clearing out the extra leftover comment lines. Another small challenge was that the original 28×28 image looked super blocky and pixelated when stretched out in Matplotlib. Adding `interpolation='bicubic'` inside `plt.imshow` smoothed out the pixel edges, making the image look clean while keeping the noise progression easy to see.

Step 5: Training Our Model

Assigned to: Fifehanmi Ogedengbe

Now we'll teach our AI to generate images. This process:

1. Takes a clear image
2. Adds random noise to it
3. Asks our AI to predict what noise was added
4. Helps our AI learn from its mistakes

This will take a while, but we'll see progress as it learns!

```
# Define helper functions needed for training and evaluation
def validate_model_parameters(model):
    """
    Counts model parameters and estimates memory usage.
    """
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

    print(f"Total parameters: {total_params:,}")
    print(f"Trainable parameters: {trainable_params:,}")

    # Estimate memory requirements (very approximate)
    param_memory = total_params * 4 / (1024 ** 2) # MB for params (float32)
    grad_memory = trainable_params * 4 / (1024 ** 2) # MB for gradients
    buffer_memory = param_memory * 2 # Optimizer state, forward activations, etc.

    print(f"Estimated GPU memory usage: {param_memory + grad_memory + buffer_memory:.1f} MB")

# Define helper functions for verifying data ranges
def verify_data_range(dataloader, name="Dataset"):
    """
```

```

Verifies the range and integrity of the data.
"""
batch = next(iter(dataloader))[0]
print(f"\n{name} range check:")
print(f"Shape: {batch.shape}")
print(f"Data type: {batch.dtype}")
print(f"Min value: {batch.min().item():.2f}")
print(f"Max value: {batch.max().item():.2f}")
print(f"Contains NaN: {torch.isnan(batch).any().item()}")
print(f"Contains Inf: {torch.isinf(batch).any().item()}")

# Define helper functions for generating samples during training
def generate_samples(model, n_samples=10):
    """
    Generates sample images using the model for visualization during training.
    """
    model.eval()
    with torch.no_grad():
        # Generate digits 0-9 for visualization
        samples = []
        for digit in range(min(n_samples, 10)):
            # Start with random noise
            x = torch.randn(1, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)

            # Set up conditioning for the digit
            c = torch.tensor([digit]).to(device)
            c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)
            c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

            # Remove noise step by step
            for t in range(n_steps-1, -1, -1):
                t_batch = torch.full((1,), t).to(device)
                x = remove_noise(x, t_batch, model, c_one_hot, c_mask)

            samples.append(x)

        # Combine samples and display
        samples = torch.cat(samples, dim=0)
        grid = make_grid(samples, nrow=min(n_samples, 5), normalize=True)

        plt.figure(figsize=(10, 4))

        # Display based on channel configuration
        if IMG_CH == 1:
            plt.imshow(grid[0].cpu(), cmap='gray')
        else:
            plt.imshow(grid.permute(1, 2, 0).cpu())

        plt.axis('off')
        plt.title('Generated Samples')
        plt.show()

# Define helper functions for safely saving models
def safe_save_model(model, path, optimizer=None, epoch=None, best_loss=None):
    """
    Safely saves model with error handling and backup.
    """
    try:
        # Create a dictionary with all the elements to save
        save_dict = {
            'model_state_dict': model.state_dict(),
        }

        # Add optional elements if provided
        if optimizer is not None:
            save_dict['optimizer_state_dict'] = optimizer.state_dict()
        if epoch is not None:
            save_dict['epoch'] = epoch
        if best_loss is not None:
            save_dict['best_loss'] = best_loss

        # Create a backup of previous checkpoint if it exists
        if os.path.exists(path):
            backup_path = path + '.backup'
            try:
                os.replace(path, backup_path)
            except:
                print(f"Created backup at {backup_path}")

```

```

except Exception as e:
    print(f"Warning: Could not create backup - {e}")

# Save the new checkpoint
torch.save(save_dict, path)
print(f"Model successfully saved to {path}")

except Exception as e:
    print(f"Error saving model: {e}")
    print("Attempting emergency save...")

    try:
        emergency_path = path + '.emergency'
        torch.save(model.state_dict(), emergency_path)
        print(f"Emergency save successful: {emergency_path}")
    except:
        print("Emergency save failed. Could not save model.")

# Create our model and move it to GPU if available
model = UNet(
    T=n_steps,                # Number of diffusion time steps
    img_ch=IMG_CH,            # Number of channels in our images (1 for grayscale, 3 for RGB)
    img_size=IMG_SIZE,        # Size of input images (28 for MNIST, 32 for CIFAR-10)
    down_chs=(32, 64, 128),    # Channel dimensions for each downsampling level
    t_embed_dim=8,            # Dimension for time step embeddings
    c_embed_dim=N_CLASSES     # Number of classes for conditioning
).to(device)

# Print model summary
print(f"\n{'='*50}")
print(f"MODEL ARCHITECTURE SUMMARY")
print(f"{'='*50}")
print(f"Input resolution: {IMG_SIZE}x{IMG_SIZE}")
print(f"Input channels: {IMG_CH}")
print(f"Time steps: {n_steps}")
print(f"Condition classes: {N_CLASSES}")
print(f"GPU acceleration: {'Yes' if device.type == 'cuda' else 'No'}")

# Validate model parameters and estimate memory requirements
# Hint: Create functions to count parameters and estimate memory usage

# Enter your code here:
validate_model_parameters(model)

# Your code to verify data ranges and integrity
# Hint: Create functions to check data ranges in training and validation data

# Enter your code here:
verify_data_range(train_dataloader, "Training data")
verify_data_range(val_dataloader, "Validation data")

# Set up the optimizer with parameters tuned for diffusion models
# Note: Lower learning rates tend to work better for diffusion models
initial_lr = 0.001 # Starting learning rate
weight_decay = 1e-5 # L2 regularization to prevent overfitting

optimizer = Adam(
    model.parameters(),
    lr=initial_lr,
    weight_decay=weight_decay
)

# Learning rate scheduler to reduce LR when validation loss plateaus
# This helps fine-tune the model toward the end of training
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',                # Reduce LR when monitored value stops decreasing
    factor=0.5,                # Multiply LR by this factor
    patience=5,                # Number of epochs with no improvement after which LR will be reduced
    min_lr=1e-6                # Lower bound on the learning rate
)

# STUDENT EXPERIMENT:
# Try different channel configurations and see how they affect:
# 1. Model size (parameter count)
# 2. Training time

```

```
# 3. Generated image quality
#
# Suggestions:
# - Smaller: down_chs=(16, 32, 64)
# - Larger: down_chs=(64, 128, 256, 512)
```

```
Created DownBlock: in_chs=32, out_chs=64, spatial_reduction=2x
Created DownBlock: in_chs=64, out_chs=128, spatial_reduction=2x
Created UpBlock: in_chs=128, out_chs=64, spatial_increase=2x
Created UpBlock: in_chs=64, out_chs=32, spatial_increase=2x
Created UNet with 3 scale levels
Channel dimensions: (32, 64, 128)
```

```
=====
MODEL ARCHITECTURE SUMMARY
=====
Input resolution: 28x28
Input channels: 1
Time steps: 100
Condition classes: 10
GPU acceleration: Yes
Total parameters: 1,598,145
Trainable parameters: 1,598,145
Estimated GPU memory usage: 24.4 MB
```

```
Training data range check:
Shape: torch.Size([64, 1, 28, 28])
Data type: torch.float32
Min value: -1.00
Max value: 1.00
Contains NaN: False
Contains Inf: False
```

```
Validation data range check:
Shape: torch.Size([64, 1, 28, 28])
Data type: torch.float32
Min value: -1.00
Max value: 1.00
Contains NaN: False
Contains Inf: False
```

```
# Implementation of the training step function
def train_step(x, c):
    """
    Performs a single training step for the diffusion model.

    This function:
    1. Prepares class conditioning
    2. Samples random timesteps for each image
    3. Adds corresponding noise to the images
    4. Asks the model to predict the noise
    5. Calculates the loss between predicted and actual noise

    Args:
        x (torch.Tensor): Batch of clean images [batch_size, channels, height, width]
        c (torch.Tensor): Batch of class labels [batch_size]

    Returns:
        torch.Tensor: Mean squared error loss value
    """
    # Convert number labels to one-hot encoding for class conditioning
    # Example: Label 3 -> [0, 0, 0, 1, 0, 0, 0, 0, 0, 0] for MNIST
    c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)

    # Create conditioning mask (all ones for standard training)
    # This would be used for classifier-free guidance if implemented
    c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

    # Pick random timesteps for each image in the batch
    # Different timesteps allow the model to learn the entire diffusion process
    t = torch.randint(0, n_steps, (x.shape[0],)).to(device)

    # Add noise to images according to the forward diffusion process
    # This simulates images at different stages of the diffusion process
    # Hint: Use the add_noise function you defined earlier

    # Enter your code here:
    x_t, noise = add_noise(x, t)
```

```

# The model tries to predict the exact noise that was added
# This is the core learning objective of diffusion models
predicted_noise = model(x_t, t, c_one_hot, c_mask)

# Calculate loss: how accurately did the model predict the noise?
# MSE loss works well for image-based diffusion models
# Hint: Use F.mse_loss to compare predicted and actual noise

# Enter your code here:
loss = F.mse_loss(predicted_noise, noise)
return loss

```

```

# Implementation of the main training loop
# Training configuration
early_stopping_patience = 10 # Number of epochs without improvement before stopping
gradient_clip_value = 1.0     # Maximum gradient norm for stability
display_frequency = 100      # How often to show progress (in steps)
generate_frequency = 500     # How often to generate samples (in steps)

# Progress tracking variables
best_loss = float('inf')
train_losses = []
val_losses = []
no_improve_epochs = 0

# Training loop
print("\n" + "="*50)
print("STARTING TRAINING")
print("="*50)
model.train()

# Wrap the training loop in a try-except block for better error handling:
# Your code for the training loop
# Hint: Use a try-except block for better error handling
# Process each epoch and each batch, with validation after each epoch

# Enter your code here:

try:
    for epoch in range(EPOCHS):
        print(f"\nEpoch {epoch+1}/{EPOCHS}")
        print("-" * 20)

        # Training phase
        model.train()
        epoch_losses = []

        # Process each batch
        for step, (images, labels) in enumerate(train_dataloader): # Fixed: dataloader → train_dataloader
            images = images.to(device)
            labels = labels.to(device)

            # Training step
            optimizer.zero_grad()
            loss = train_step(images, labels)
            loss.backward()

            # Add gradient clipping for stability
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=gradient_clip_value)

            optimizer.step()
            epoch_losses.append(loss.item())

        # Show progress at regular intervals
        if step % display_frequency == 0:
            print(f" Step {step}/{len(train_dataloader)}, Loss: {loss.item():.4f}")

            # Generate samples less frequently to save time
            if step % generate_frequency == 0 and step > 0:
                print(" Generating samples...")
                generate_samples(model, n_samples=5)

        # End of epoch - calculate average training loss
        avg_train_loss = sum(epoch_losses) / len(epoch_losses)
        train_losses.append(avg_train_loss)

```

```

print(f"\nTraining - Epoch {epoch+1} average loss: {avg_train_loss:.4f}")

# Validation phase
model.eval()
val_epoch_losses = []
print("Running validation...")

with torch.no_grad(): # Disable gradients for validation
    for val_images, val_labels in val_dataloader:
        val_images = val_images.to(device)
        val_labels = val_labels.to(device)

        # Calculate validation loss
        val_loss = train_step(val_images, val_labels)
        val_epoch_losses.append(val_loss.item())

# Calculate average validation loss
avg_val_loss = sum(val_epoch_losses) / len(val_epoch_losses)
val_losses.append(avg_val_loss)
print(f"Validation - Epoch {epoch+1} average loss: {avg_val_loss:.4f}")

# Learning rate scheduling based on validation loss
scheduler.step(avg_val_loss)
current_lr = optimizer.param_groups[0]['lr']
print(f"Learning rate: {current_lr:.6f}")

# Generate samples at the end of each epoch
if epoch % 2 == 0 or epoch == EPOCHS - 1:
    print("\nGenerating samples for visual progress check...")
    generate_samples(model, n_samples=10)

# Save best model based on validation loss
if avg_val_loss < best_loss:
    best_loss = avg_val_loss
    # Use safe_save_model instead of just saving state_dict
    safe_save_model(model, 'best_diffusion_model.pt', optimizer, epoch, best_loss)
    print(f"✓ New best model saved! (Val Loss: {best_loss:.4f})")
    no_improve_epochs = 0
else:
    no_improve_epochs += 1
    print(f"No improvement for {no_improve_epochs}/{early_stopping_patience} epochs")

# Early stopping
if no_improve_epochs >= early_stopping_patience:
    print("\nEarly stopping triggered! No improvement in validation loss.")
    break

# Plot loss curves every few epochs
if epoch % 5 == 0 or epoch == EPOCHS - 1:
    plt.figure(figsize=(10, 5))
    plt.plot(train_losses, label='Training Loss')
    plt.plot(val_losses, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid(True)
    plt.show()
except KeyboardInterrupt:
    print("Training interrupted by user.")
    safe_save_model(model, 'interrupted_model.pt', optimizer, epoch, best_loss)
except RuntimeError as e:
    if "out of memory" in str(e):
        print("GPU out of memory - try lowering BATCH_SIZE.")
        torch.cuda.empty_cache()
    else:
        raise

# Final wrap-up
print("\n" + "="*50)
print("TRAINING COMPLETE")
print("="*50)
print(f"Best validation loss: {best_loss:.4f}")

```

```
# Generate final samples
print("Generating final samples...")
generate_samples(model, n_samples=10)
model.train()

# Display final loss curves
plt.figure(figsize=(12, 5))
plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid(True)
plt.show()

# Clean up memory
torch.cuda.empty_cache()
```

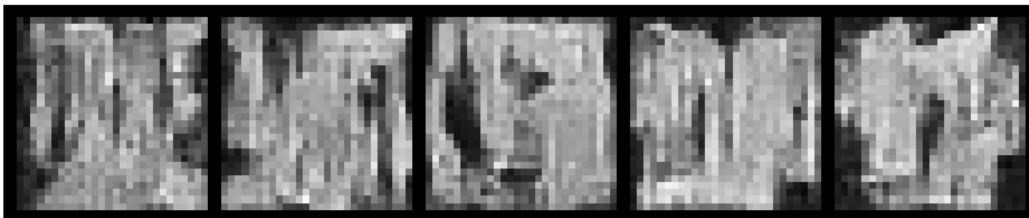


```
=====
STARTING TRAINING
=====
```

Epoch 1/30

```
-----
Step 0/750, Loss: 1.0849
Step 100/750, Loss: 0.1782
Step 200/750, Loss: 0.1730
Step 300/750, Loss: 0.2010
Step 400/750, Loss: 0.1424
Step 500/750, Loss: 0.1640
Generating samples...
```

Generated Samples



```
Step 600/750, Loss: 0.1451
Step 700/750, Loss: 0.1843
```

```
Training - Epoch 1 average loss: 0.1874
Running validation...
Validation - Epoch 1 average loss: 0.1510
Learning rate: 0.001000
```

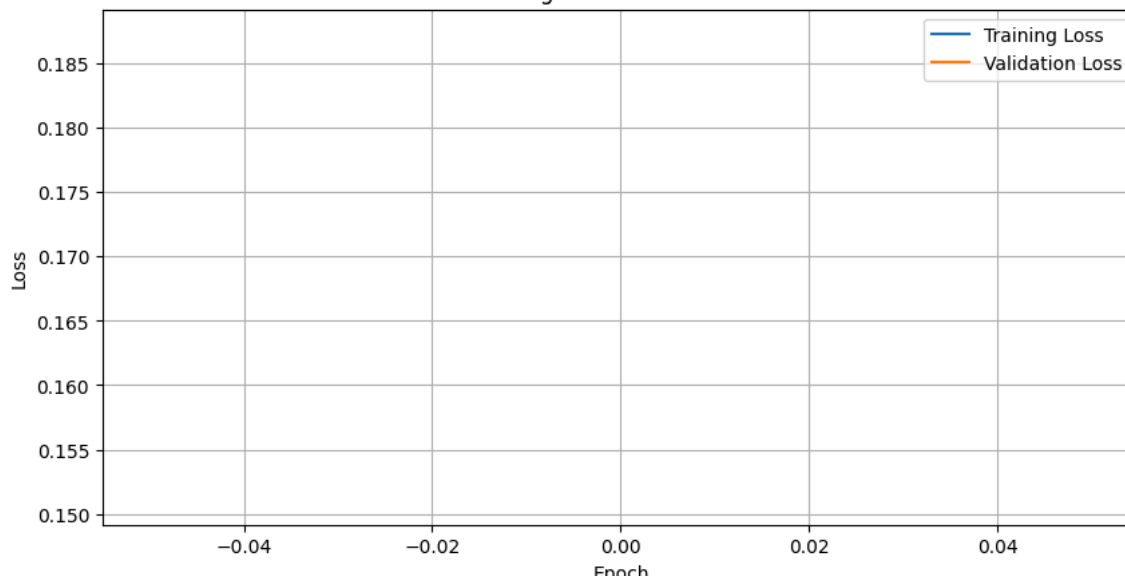
Generating samples for visual progress check...

Generated Samples



```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
✓ New best model saved! (Val Loss: 0.1510)
```

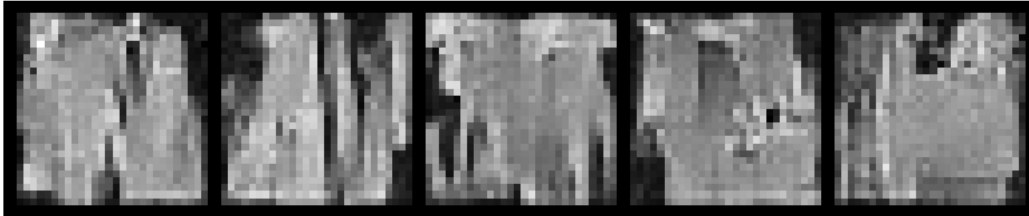
Training and Validation Loss



Epoch 2/30

Step 0/750, Loss: 0.1654
Step 100/750, Loss: 0.1644
Step 200/750, Loss: 0.1315
Step 300/750, Loss: 0.1301
Step 400/750, Loss: 0.1690
Step 500/750, Loss: 0.1294
Generating samples...

Generated Samples



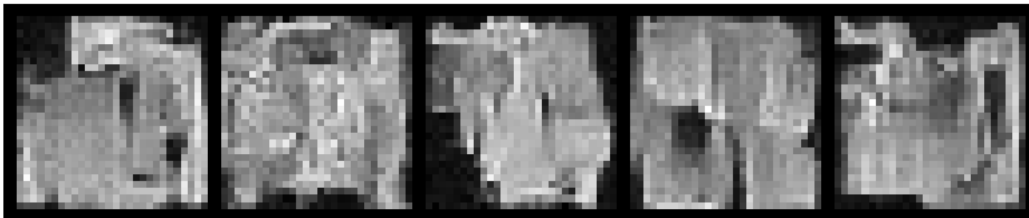
Step 600/750, Loss: 0.1522
Step 700/750, Loss: 0.1308

Training - Epoch 2 average loss: 0.1443
Running validation...
Validation - Epoch 2 average loss: 0.1419
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
✓ New best model saved! (Val Loss: 0.1419)

Epoch 3/30

Step 0/750, Loss: 0.1325
Step 100/750, Loss: 0.1315
Step 200/750, Loss: 0.1203
Step 300/750, Loss: 0.1451
Step 400/750, Loss: 0.1526
Step 500/750, Loss: 0.1508
Generating samples...

Generated Samples



Step 600/750, Loss: 0.1260
Step 700/750, Loss: 0.1237

Training - Epoch 3 average loss: 0.1353
Running validation...
Validation - Epoch 3 average loss: 0.1335
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



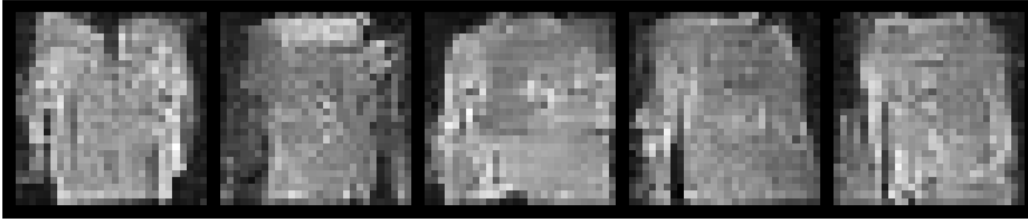
Created backup at best_diffusion_model.pt.backup

Model successfully saved to best_diffusion_model.pt
✓ New best model saved! (Val Loss: 0.1335)

Epoch 4/30

Step 0/750, Loss: 0.1242
Step 100/750, Loss: 0.1294
Step 200/750, Loss: 0.1289
Step 300/750, Loss: 0.1479
Step 400/750, Loss: 0.1139
Step 500/750, Loss: 0.1372
Generating samples...

Generated Samples



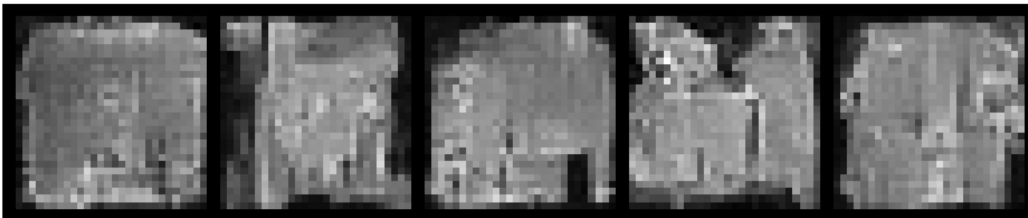
Step 600/750, Loss: 0.1172
Step 700/750, Loss: 0.1246

Training - Epoch 4 average loss: 0.1306
Running validation...
Validation - Epoch 4 average loss: 0.1287
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
✓ New best model saved! (Val Loss: 0.1287)

Epoch 5/30

Step 0/750, Loss: 0.1065
Step 100/750, Loss: 0.1211
Step 200/750, Loss: 0.1296
Step 300/750, Loss: 0.1364
Step 400/750, Loss: 0.1298
Step 500/750, Loss: 0.1296
Generating samples...

Generated Samples

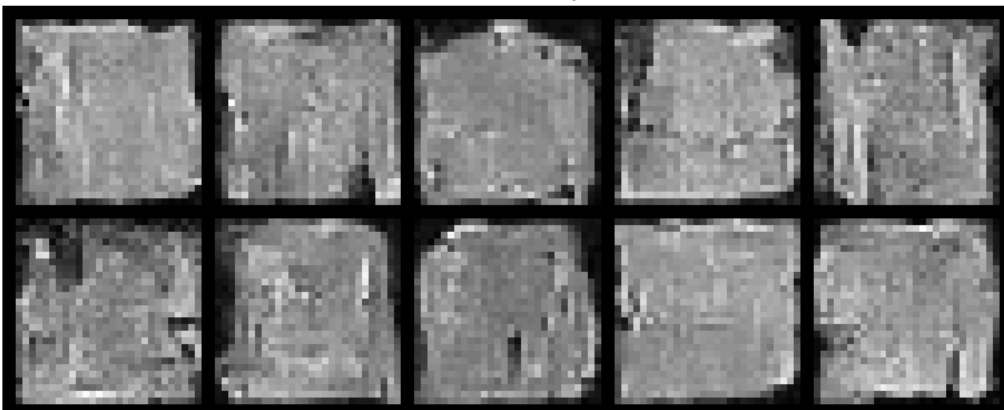


Step 600/750, Loss: 0.1190
Step 700/750, Loss: 0.1292

Training - Epoch 5 average loss: 0.1261
Running validation...
Validation - Epoch 5 average loss: 0.1251
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples

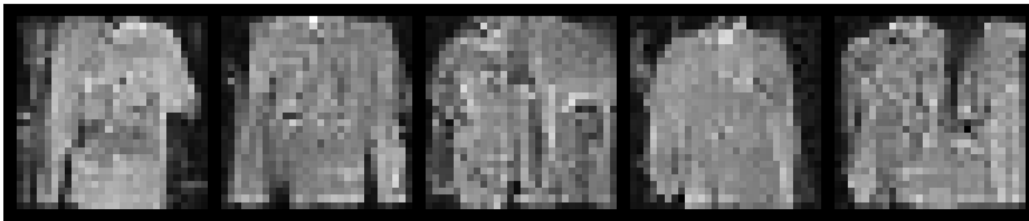


Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
✓ New best model saved! (Val Loss: 0.1251)

Epoch 6/30

Step 0/750, Loss: 0.1389
Step 100/750, Loss: 0.1223
Step 200/750, Loss: 0.1353
Step 300/750, Loss: 0.1242
Step 400/750, Loss: 0.1188
Step 500/750, Loss: 0.1178
Generating samples...

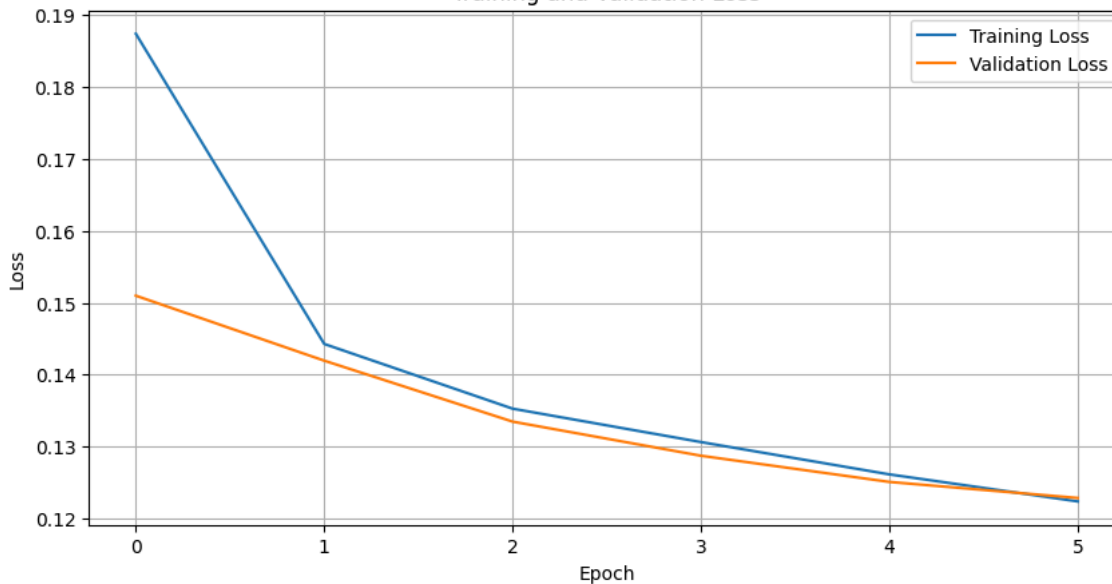
Generated Samples



Step 600/750, Loss: 0.1111
Step 700/750, Loss: 0.1150

Training - Epoch 6 average loss: 0.1224
Running validation...
Validation - Epoch 6 average loss: 0.1229
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
✓ New best model saved! (Val Loss: 0.1229)

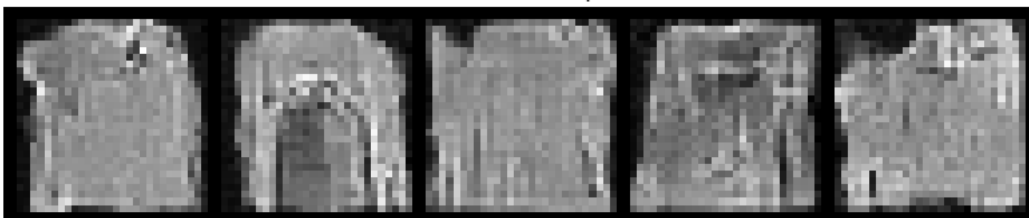
Training and Validation Loss



Epoch 7/30

Step 0/750, Loss: 0.1045
Step 100/750, Loss: 0.1146
Step 200/750, Loss: 0.1070
Step 300/750, Loss: 0.1167
Step 400/750, Loss: 0.1273
Step 500/750, Loss: 0.1005
Generating samples...

Generated Samples



Step 600/750, Loss: 0.1071

Step 700/750, Loss: 0.1090

Training - Epoch 7 average loss: 0.1205

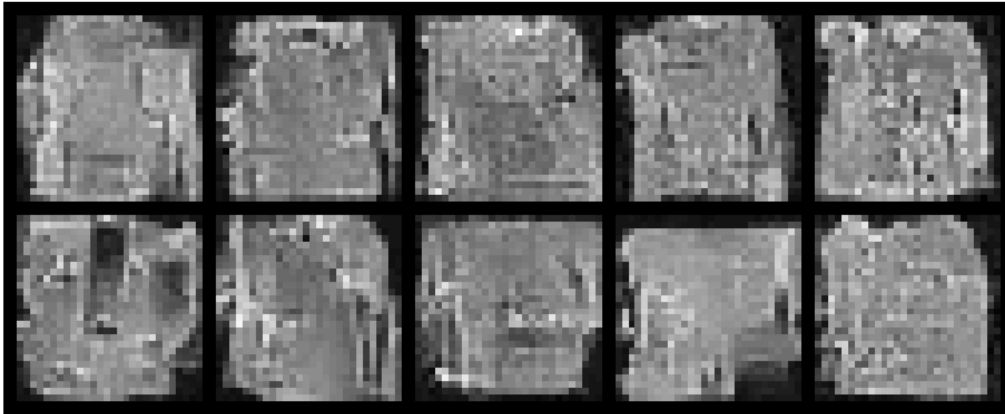
Running validation...

Validation - Epoch 7 average loss: 0.1191

Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
✓ New best model saved! (Val Loss: 0.1191)

Epoch 8/30

Step 0/750, Loss: 0.1360

Step 100/750, Loss: 0.0886

Step 200/750, Loss: 0.1337

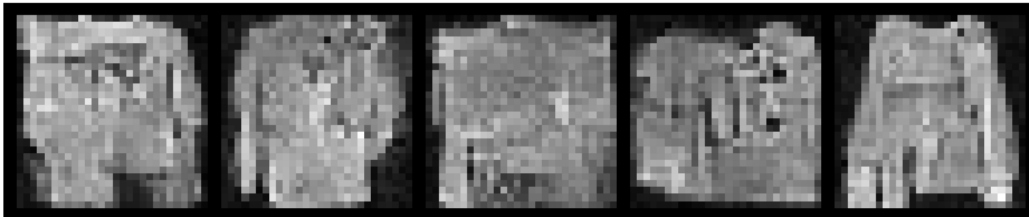
Step 300/750, Loss: 0.1012

Step 400/750, Loss: 0.1337

Step 500/750, Loss: 0.1265

Generating samples...

Generated Samples



Step 600/750, Loss: 0.1318

Step 700/750, Loss: 0.1119

Training - Epoch 8 average loss: 0.1191

Running validation...

Validation - Epoch 8 average loss: 0.1185

Learning rate: 0.001000

Created backup at best_diffusion_model.pt.backup

Model successfully saved to best_diffusion_model.pt

✓ New best model saved! (Val Loss: 0.1185)

Epoch 9/30

Step 0/750, Loss: 0.1097

Step 100/750, Loss: 0.1042

Step 200/750, Loss: 0.1025

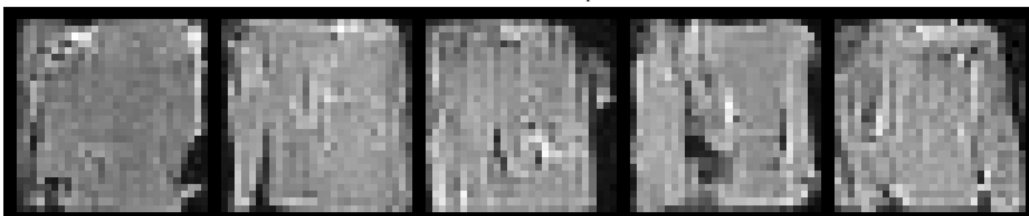
Step 300/750, Loss: 0.1052

Step 400/750, Loss: 0.1203

Step 500/750, Loss: 0.1311

Generating samples...

Generated Samples

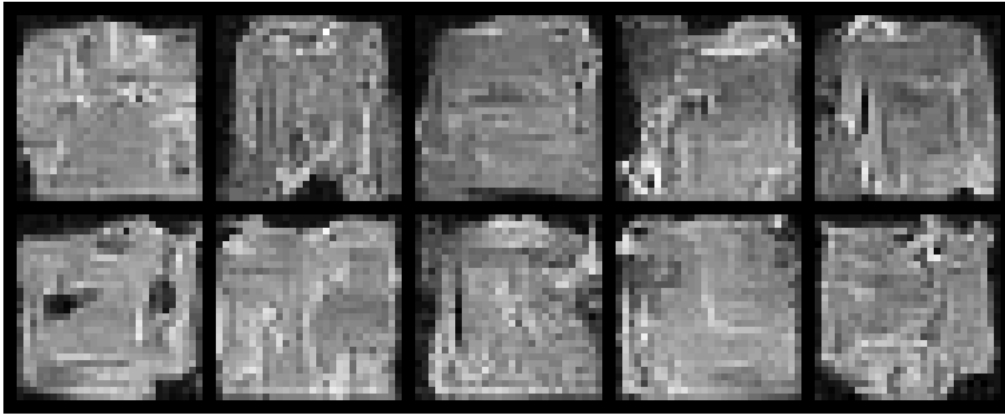


Step 600/750, Loss: 0.1058
 Step 700/750, Loss: 0.1342

Training - Epoch 9 average loss: 0.1181
 Running validation...
 Validation - Epoch 9 average loss: 0.1163
 Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples

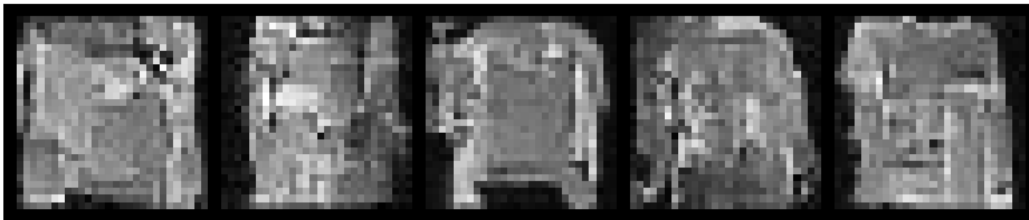


Created backup at best_diffusion_model.pt.backup
 Model successfully saved to best_diffusion_model.pt
 ✓ New best model saved! (Val Loss: 0.1163)

Epoch 10/30

 Step 0/750, Loss: 0.1147
 Step 100/750, Loss: 0.1238
 Step 200/750, Loss: 0.1288
 Step 300/750, Loss: 0.1213
 Step 400/750, Loss: 0.1402
 Step 500/750, Loss: 0.1249
 Generating samples...

Generated Samples



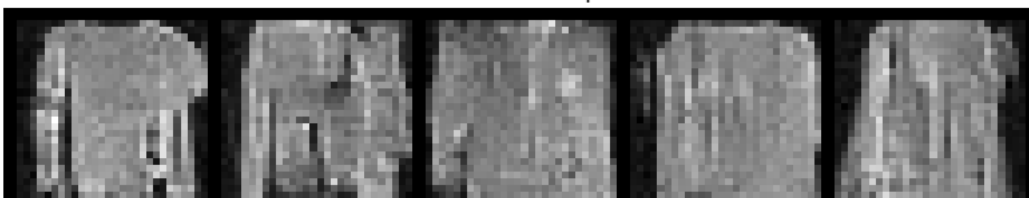
Step 600/750, Loss: 0.1000
 Step 700/750, Loss: 0.1239

Training - Epoch 10 average loss: 0.1168
 Running validation...
 Validation - Epoch 10 average loss: 0.1160
 Learning rate: 0.001000
 Created backup at best_diffusion_model.pt.backup
 Model successfully saved to best_diffusion_model.pt
 ✓ New best model saved! (Val Loss: 0.1160)

Epoch 11/30

 Step 0/750, Loss: 0.1262
 Step 100/750, Loss: 0.1189
 Step 200/750, Loss: 0.1017
 Step 300/750, Loss: 0.0970
 Step 400/750, Loss: 0.1014
 Step 500/750, Loss: 0.1204
 Generating samples...

Generated Samples

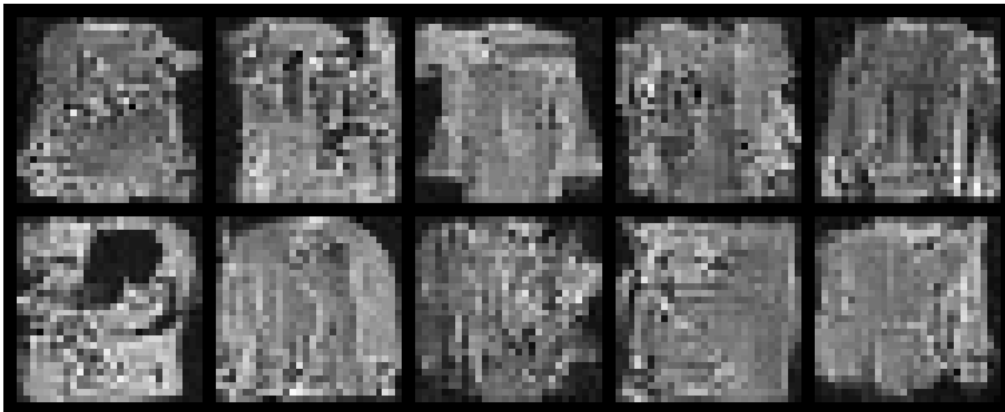


```
Step 600/750, Loss: 0.0999
Step 700/750, Loss: 0.1064
```

```
Training - Epoch 11 average loss: 0.1158
Running validation...
Validation - Epoch 11 average loss: 0.1142
Learning rate: 0.001000
```

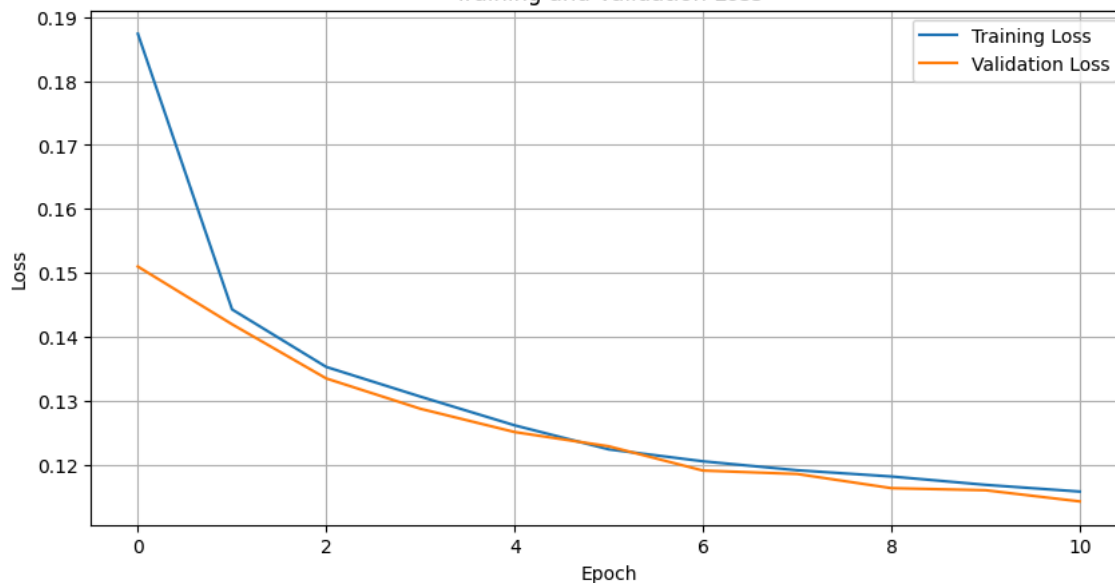
Generating samples for visual progress check...

Generated Samples



```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
✓ New best model saved! (Val Loss: 0.1142)
```

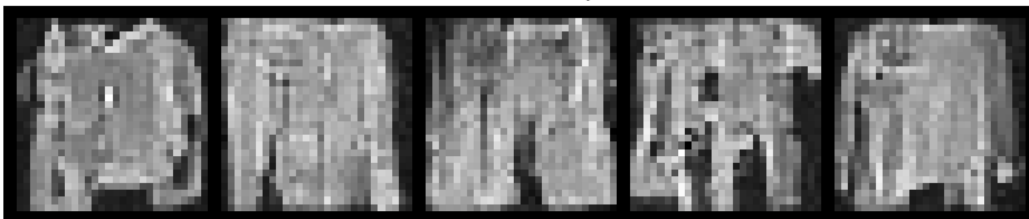
Training and Validation Loss



Epoch 12/30

```
-----
Step 0/750, Loss: 0.0926
Step 100/750, Loss: 0.0924
Step 200/750, Loss: 0.1230
Step 300/750, Loss: 0.1191
Step 400/750, Loss: 0.1161
Step 500/750, Loss: 0.1235
Generating samples...
```

Generated Samples



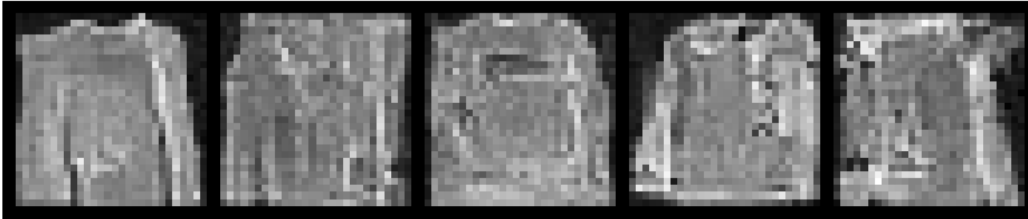
```
Step 600/750, Loss: 0.1362
Step 700/750, Loss: 0.1128
```

Training - Epoch 12 average loss: 0.1146
Running validation...
Validation - Epoch 12 average loss: 0.1144
Learning rate: 0.001000
No improvement for 1/10 epochs

Epoch 13/30

Step 0/750, Loss: 0.1113
Step 100/750, Loss: 0.1056
Step 200/750, Loss: 0.1357
Step 300/750, Loss: 0.0957
Step 400/750, Loss: 0.1331
Step 500/750, Loss: 0.1347
Generating samples...

Generated Samples

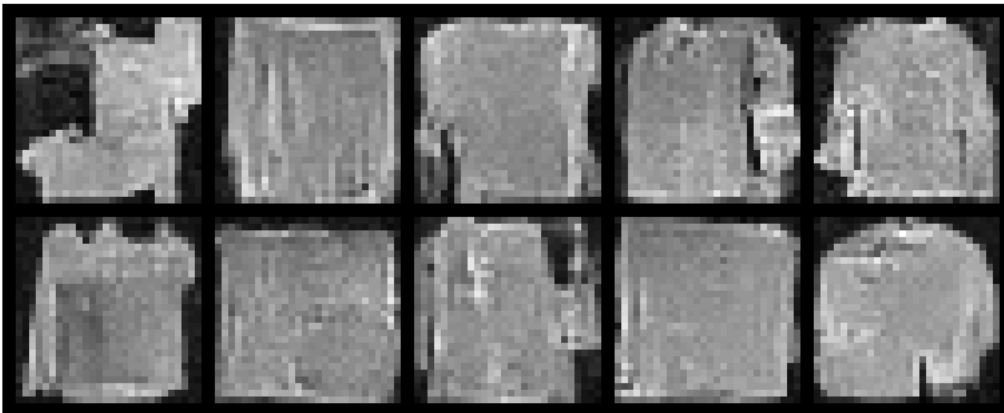


Step 600/750, Loss: 0.1056
Step 700/750, Loss: 0.1356

Training - Epoch 13 average loss: 0.1138
Running validation...
Validation - Epoch 13 average loss: 0.1148
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples

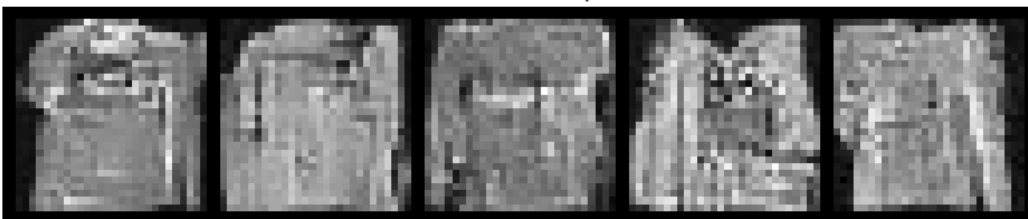


No improvement for 2/10 epochs

Epoch 14/30

Step 0/750, Loss: 0.1257
Step 100/750, Loss: 0.1288
Step 200/750, Loss: 0.1129
Step 300/750, Loss: 0.1116
Step 400/750, Loss: 0.1251
Step 500/750, Loss: 0.1231
Generating samples...

Generated Samples



Step 600/750, Loss: 0.1015
Step 700/750, Loss: 0.0828

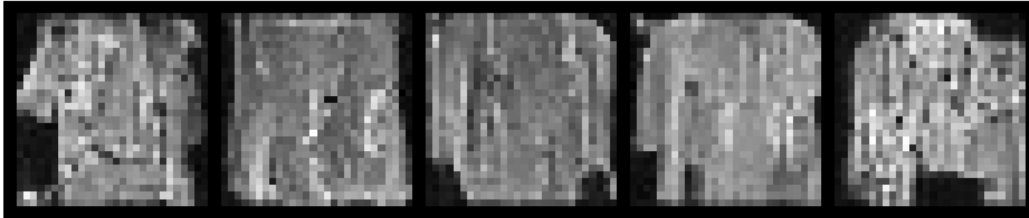
Training - Epoch 14 average loss: 0.1141
Running validation...

Validation - Epoch 14 average loss: 0.1155
Learning rate: 0.001000
No improvement for 3/10 epochs

Epoch 15/30

Step 0/750, Loss: 0.1021
Step 100/750, Loss: 0.1256
Step 200/750, Loss: 0.1122
Step 300/750, Loss: 0.1032
Step 400/750, Loss: 0.1289
Step 500/750, Loss: 0.1204
Generating samples...

Generated Samples

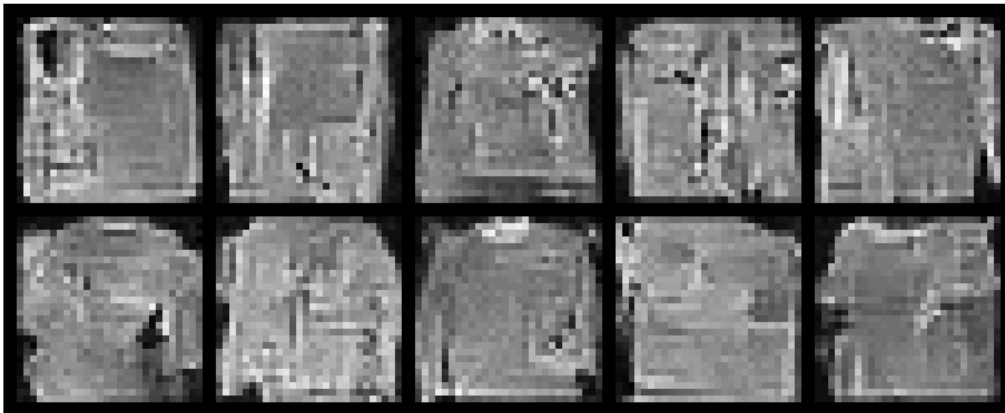


Step 600/750, Loss: 0.1133
Step 700/750, Loss: 0.1101

Training - Epoch 15 average loss: 0.1130
Running validation...
Validation - Epoch 15 average loss: 0.1148
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples

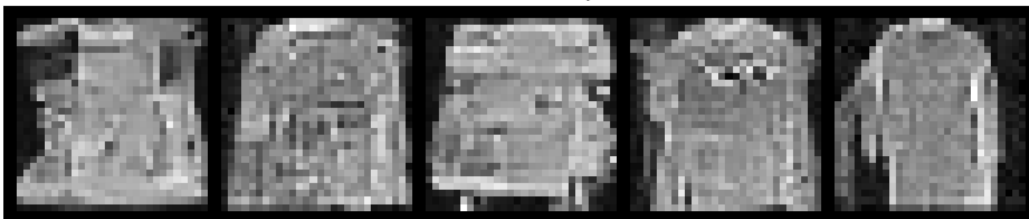


No improvement for 4/10 epochs

Epoch 16/30

Step 0/750, Loss: 0.1121
Step 100/750, Loss: 0.0970
Step 200/750, Loss: 0.0979
Step 300/750, Loss: 0.0921
Step 400/750, Loss: 0.1258
Step 500/750, Loss: 0.1006
Generating samples...

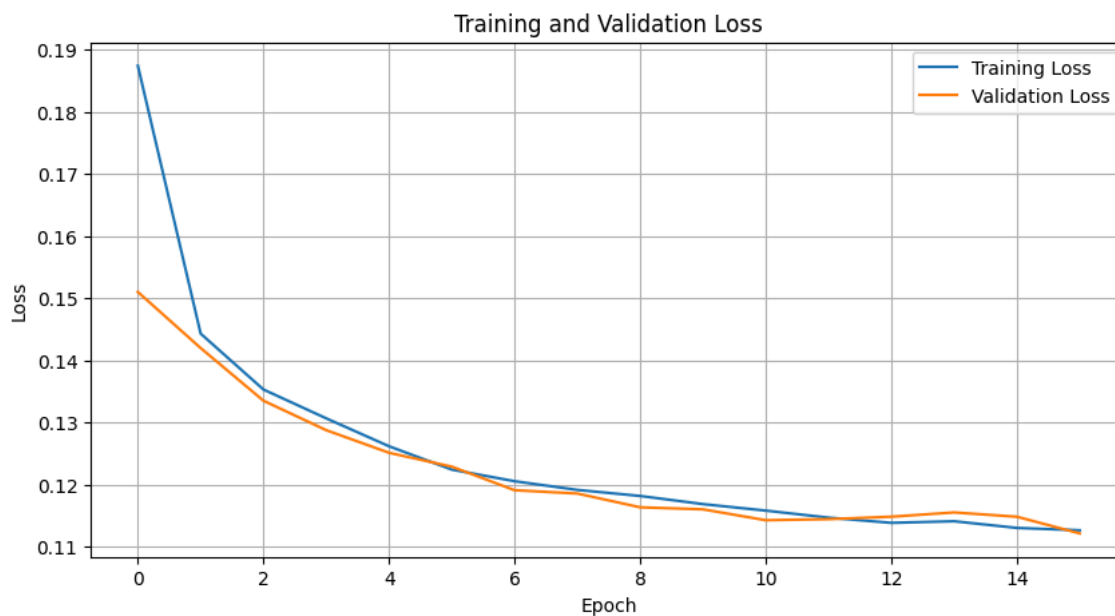
Generated Samples



Step 600/750, Loss: 0.1168
Step 700/750, Loss: 0.1233

Training - Epoch 16 average loss: 0.1126
Running validation...
Validation - Epoch 16 average loss: 0.1121
Learning rate: 0.001000
Created backup at best diffusion model.pt.backup

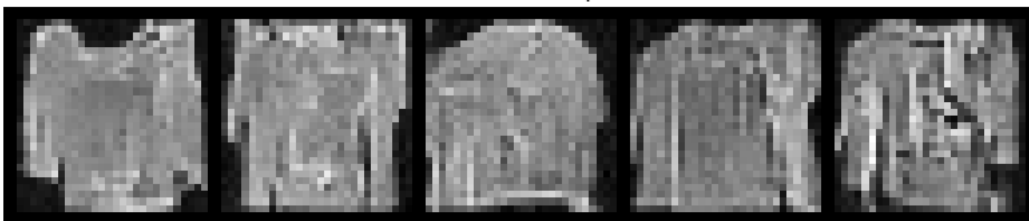
Model successfully saved to best_diffusion_model.pt
 ✓ New best model saved! (Val Loss: 0.1121)



Epoch 17/30

 Step 0/750, Loss: 0.0987
 Step 100/750, Loss: 0.1077
 Step 200/750, Loss: 0.1147
 Step 300/750, Loss: 0.0924
 Step 400/750, Loss: 0.1272
 Step 500/750, Loss: 0.1337
 Generating samples...

Generated Samples

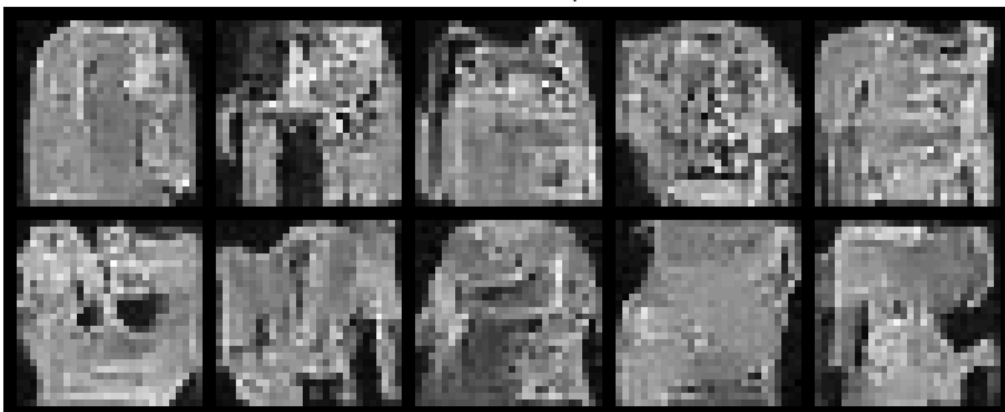


Step 600/750, Loss: 0.1398
 Step 700/750, Loss: 0.1028

Training - Epoch 17 average loss: 0.1121
 Running validation...
 Validation - Epoch 17 average loss: 0.1107
 Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples

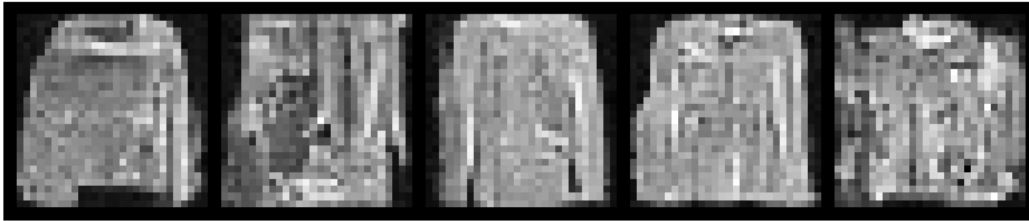


Created backup at best_diffusion_model.pt.backup
 Model successfully saved to best_diffusion_model.pt
 ✓ New best model saved! (Val Loss: 0.1107)

Epoch 18/30

Step 0/750, Loss: 0.1133
Step 100/750, Loss: 0.0826
Step 200/750, Loss: 0.0966
Step 300/750, Loss: 0.0972
Step 400/750, Loss: 0.1225
Step 500/750, Loss: 0.1268
Generating samples...

Generated Samples



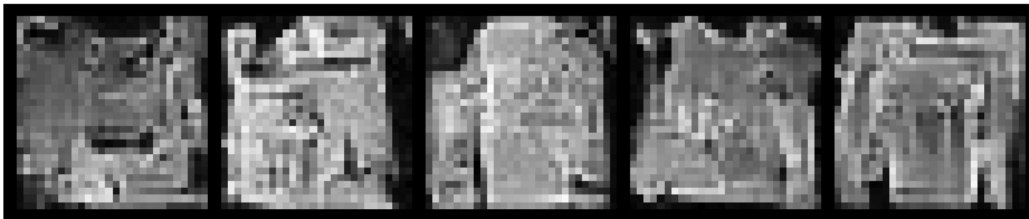
Step 600/750, Loss: 0.1051
Step 700/750, Loss: 0.0864

Training - Epoch 18 average loss: 0.1110
Running validation...
Validation - Epoch 18 average loss: 0.1108
Learning rate: 0.001000
No improvement for 1/10 epochs

Epoch 19/30

Step 0/750, Loss: 0.1330
Step 100/750, Loss: 0.1513
Step 200/750, Loss: 0.1095
Step 300/750, Loss: 0.1089
Step 400/750, Loss: 0.1279
Step 500/750, Loss: 0.1084
Generating samples...

Generated Samples

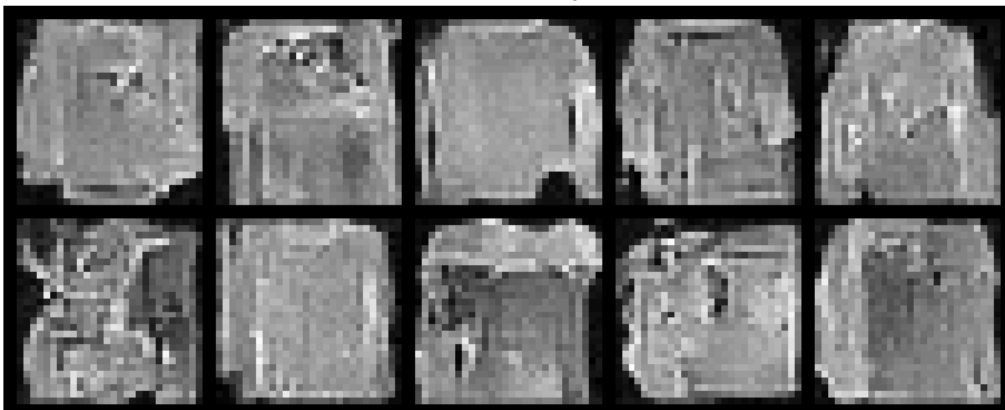


Step 600/750, Loss: 0.0944
Step 700/750, Loss: 0.1453

Training - Epoch 19 average loss: 0.1109
Running validation...
Validation - Epoch 19 average loss: 0.1122
Learning rate: 0.001000

Generating samples for visual progress check...

Generated Samples



No improvement for 2/10 epochs

Epoch 20/30

Step 0/750, Loss: 0.1278